

UTN San Nicolás - Tecnicatura En Programación A Distancia

Comisión 4 - Pichulman Miguel

TPI Integrador Programacion III

30/6/26

Índice

Frontend.....	3
Marco Teórico	3
Decisiones Técnicas y Arquitectura.....	4
Arquitectura Frontend	4
Persistencia Temporal y Mocking de Datos	4
Dificultades y Soluciones.....	5
Conflictos de Resolución de Dependencias (NPM):	5
Inconsistencias en el Modelo de Estados:.....	5
Backend.....	6
Marco Teórico	6
Decisiones Técnicas y Arquitectura.....	7
Dificultades y Soluciones.....	8

Frontend

Marco Teórico

Para el desarrollo de la plataforma "Food Store", se optó por un stack tecnológico moderno que permite separar las responsabilidades del lado del cliente y del servidor, garantizando escalabilidad y mantenimiento.

Vite & TypeScript: Se utilizó Vite como herramienta de construcción (bundler) por su extrema rapidez en el entorno de desarrollo. TypeScript se implementó para añadir tipado estático a JavaScript, lo cual previene errores en tiempo de ejecución y facilita la manipulación estricta de las estructuras de datos (como el modelo de Pedidos y Productos).

HTML5 y CSS3 puro: El diseño de la interfaz se mantuvo en un nivel intermedio, priorizando la claridad, la semántica de las etiquetas y una experiencia de usuario fluida sin depender de frameworks CSS pesados.

Decisiones Técnicas y Arquitectura

Arquitectura Frontend

El sistema cliente se desarrolló bajo una arquitectura MPA (Multiple Page Application). A diferencia de las SPA (Single Page Applications), el proyecto divide funcionalmente sus módulos en distintos archivos físicos HTML (Catálogo, Carrito, Panel de Administración, etc.).

Punto de entrada: Se configuró el flujo para que el archivo index.html redirija mediante main.ts directamente al módulo de autenticación, asegurando que ningún usuario acceda al sistema sin validación de identidad.

Protección por Roles: Se implementaron "guardias" en la inicialización de los controladores (init()). Por ejemplo, si un usuario con rol ADMIN intenta acceder por URL a la ruta del carrito de compras, el script detecta el rol y realiza una redirección forzada, ocultando además los botones correspondientes en el header.

Persistencia Temporal y Mocking de Datos

Ante la ausencia temporal del backend definitivo, se construyó una capa de servicios basada en la API Fetch que consume archivos .json locales.

LocalStorage: Para emular las operaciones CRUD en el panel administrativo sin perder la integridad tras recargar la página (F5), se decidió que todas las altas, bajas lógicas y modificaciones de estados de pedidos persistan en la memoria del navegador.

Dificultades y Soluciones

Durante el desarrollo, surgieron diversos obstáculos técnicos que requirieron refactorizaciones específicas:

Conflictos de Resolución de Dependencias (NPM):

Problema: Al instalar el entorno de Vite y TypeScript, se produjeron bloqueos de tipo ERESOLVE debido a incompatibilidades de versiones entre dependencias (específicamente typescript-eslint).

Solución: Se procedió a realizar una limpieza completa del árbol de dependencias (node_modules y package-lock.json) y se forzó la instalación ignorando dependencias pares heredadas mediante el flag --legacy-peer-deps.

Inconsistencias en el Modelo de Estados:

Problema: Los datos de prueba en los archivos JSON contaban con estados de pedidos (ej. "ENTREGADO", "EN_PREPARACION") que no correspondían con el Enum oficial exigido por la rúbrica (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO). Esto rompía la lógica visual y de filtrado.

Solución: Se modificó el atributo de estado en los datos fuente para garantizar la compatibilidad estricta con el modelo de dominio oficial. El sistema trabaja exclusivamente con los estados: `PENDIENTE`, `CONFIRMADO`, `TERMINADO` y `CANCELADO`.

Backend

Marco Teórico

El presente proyecto implementa el backend del sistema "Food Store", diseñado para gestionar categorías, productos, usuarios y pedidos. Las tecnologías y conceptos clave aplicados son:

- Java SE: Lenguaje utilizado para la lógica de negocio y la interfaz de consola interactiva.
- JPA e Hibernate: Utilizados para el mapeo objeto-relacional (ORM), permitiendo interactuar con la base de datos utilizando objetos Java de manera fluida.
- Base de Datos H2: Motor relacional configurado en modo archivo. Es ideal para este entorno ya que permite que Hibernate autogestione el esquema.
- Lombok: Librería implementada para reducir el código repetitivo, generando constructores y métodos de acceso de forma automática.

Decisiones Técnicas y Arquitectura

Para resolver los requerimientos manteniendo un enfoque claro y un nivel adecuado para la etapa académica actual, se tomaron las siguientes decisiones:

- Patrón Repositorio Genérico: Se diseñó la clase `BaseRepository<T>` que centraliza las operaciones CRUD. Los repositorios específicos heredan de esta base, evitando reescribir la lógica de conexión.
- Clase Base y Herencia: Se implementó la clase abstracta `Base` para concentrar los atributos comunes a todas las tablas (como el ID y el estado de eliminación).
- Bajas Lógicas (Soft Delete): Los registros no se eliminan físicamente para proteger el historial. Se utiliza el atributo `eliminado = true` y las consultas JPQL filtran únicamente los registros activos.
- Transaccionalidad Atómica: El alta de un pedido se procesa bajo un único `EntityManager`, garantizando que la asociación del usuario, el cálculo del total y la reducción del stock ocurran como un bloque seguro.

Dificultades y Soluciones

Durante el desarrollo surgieron algunos obstáculos técnicos que se resolvieron de la siguiente manera:

- Manejo de entidades desprendidas: Al guardar un pedido con productos buscados previamente, se generaban errores al provenir de distintos contextos de persistencia. Se solucionó centralizando la lógica en un único método transaccional donde se recuperan los objetos gestionados (em.find) antes de persistir.
- Relaciones bidireccionales: Al intentar guardar los detalles del pedido, la clave foránea quedaba nula. Se implementó el método addDetallePedido que fuerza la relación inversa, asegurando el mapeo correcto con CascadeType.ALL.